

DATA ROOM — JUNE 2026

The PolarGrid Software Moat

How usage compounds into routing intelligence, and why the fleet that carries the traffic owns the map that cannot be replicated.

Infrastructure is the base. Software is the leverage.

Why usage compounds value in ways capital cannot replicate.

PolarGrid runs physical inference infrastructure: GPUs, edge nodes, and data center hardware. That base scales the way infrastructure always has, with capital. Add a node, add cost, add roughly proportional capacity.

What changes the shape of the business is the layer above it. **The software that decides where every request runs improves with usage.** Capital adds nodes; usage compounds the software, making it better, cheaper per request, and harder to leave the more traffic flows through it, at close to zero marginal cost.

This is a flywheel: traffic builds a map of real-world latency and demand, the map sharpens routing, sharper routing improves performance, and better performance brings more traffic. **As the network grows, software's share of the value rises with scale.**

The routing layer and its telemetry pipeline are **deployed today** serving production traffic across multiple regions. At almost every other provider, inference is bought as a single regional endpoint the customer pins by hand, even though region choice is one of the largest latency levers in the stack: **10 to 30 ms round-trip** to the North American population from an edge node, versus **60 to 150 ms** from a single us-east-1 region.

Automatic routing across a fleet the provider owns, which is what PolarGrid already does, is rare in a market built on static endpoints and third-party aggregators that dispatch across other providers' APIs.

Where the software captures value

Two parts of the software layer turn usage into value the infrastructure underneath cannot capture on its own.



Where value is held

PolarGrid's edge gateway is **model-aware**. A Model Registry maps each request to a node that has the right model loaded and the right backend for it, across vLLM and custom Python runtimes with TensorRT-LLM supported, spanning language, speech, vision, and multimodal.

A customer builds against the abstraction. Their workloads, model formats, and routing policies run through it, and moving off means rebuilding a serving layer. **That is where pricing power and retention live: in the software layer.**



Where per-request economics improve

The routing layer decides where each request runs, and as it accumulates data it makes that decision better. Each turn of the flywheel lowers latency and cost per token and raises utilization on the same fleet.

The customer feels this as **time to first token (text)** and **time to first audio (voice)**: the sharper the map, the lower the p50 and p99 the same hardware delivers. **The infrastructure spend is largely fixed; the value extracted from it rises with the quality of the routing.**

Each year the same nodes do more, and that gain is captured in the software, not the hardware. The per-request economics compound in the software layer without requiring additional capital spend on the infrastructure layer.

Why the lead compounds

The flywheel is not a framework. It is the mechanism by which every request widens the gap.

STEP 01

Traffic builds the map

Every routing decision is logged, capturing region, distance, handler latency, and queue depth, feeding a real-time pipeline. As request volume grows this becomes a continuously refreshed map of real-world inference latency and demand: which corridors congest, how specific models behave under load on specific hardware, where the next request will originate.

STEP 02

The map sharpens routing

Today the router is model-aware, load-aware, and geo-aware. Workload profiling is the next layer: classify each request by modality, latency class, length, and session before placement, then apply node selection, KV-cache session affinity, and SLA tier to it. The inputs are already captured in per-request telemetry.

STEP 03

Sharper routing lowers latency

Better placement decisions reduce time to first token and time to first audio on the same physical fleet. The customer experiences this as performance improvement. The provider captures it as higher utilization, lower cost per request, and stronger retention without increasing infrastructure spend.

STEP 04

Better performance brings more traffic

More traffic sharpens the map further. A competitor chases a target that pulls further ahead the longer the network runs. **The map cannot be bought, licensed, or reverse-engineered. The only way to build it is to carry the traffic.**

Deepening the layer

Every item below extends something already running in production. Each tightens one stage of the flywheel. Prioritization follows customer demand.

CAPABILITY	BUILDS ON (LIVE TODAY)	LOOP STAGE	STATUS
Predictive, demand-aware routing	Decision logging + analytics pipeline	map → routing	Live
Session-affinity (KV-cache) routing	Queue-depth-aware node selection	routing → performance	Next
Anomaly-aware failover	Health-check service + telemetry	map → routing	Next
Dynamic model placement	Model Registry + multi-backend serving	performance → traffic	Planned
SLA-tiered routing classes	Server-side routing policy + telemetry	routing → performance	Planned

Each capability listed above is grounded in infrastructure that is already instrumented and running. The inputs, the telemetry, the routing hooks, are already captured. What remains is applying accumulated data to progressively tighter placement decisions.

Why the value accrues to the software

The reasonable challenge to any software-over-infrastructure thesis is that the capacity owner underneath simply absorbs the layer.

What can be matched

A competitor can match the capacity, because that is the linear, capital part. They can add GPU nodes, negotiate similar hardware contracts, and build out regional presence. That is finite, reproducible, and purchasable.

What cannot be added

What they cannot add is the **accumulated usage the routing intelligence learns from**, or the **installed workloads already running through the abstraction**. Those are the non-linear parts, and they compound. Every customer who builds against the Model Registry, every inference workload that runs through the gateway, and every routing decision that lands in the telemetry pipeline increases the cost of switching and widens the gap in routing quality.

The distinction that holds the moat

Routing across other providers' APIs already exists. **Routing across an edge-GPU fleet PolarGrid owns, scored on a latency and demand map only owned traffic can build, does not.** The fleet earns the right to collect the data; the data is what compounds.

So as the network scales, the hardware stays a commodity others can rent, while the software layer captures a growing share of the value and gets harder to copy with every request it handles.

The fleet earns the data. The data is what compounds.

Every request handled is a routing decision logged. Every decision logged sharpens the map. The map is the moat, and the only way to build it is to carry the traffic.

CONTACT

Rade Kovacevic

rade@polargrid.ai